

Expressions, Assignment, and Control Flow

Liting Zhang

University of West Florida

Overviews

- Expressions: evaluation order, side effects, short-circuit
- Assignment: statements vs expressions, compound assignment, destructuring
- Sequences and blocks: sequencing, scoping, readability
- Selection: if/else, switch, match (TypeScript, C, Python, Rust)
- Iteration: while, for, foreach, iterators and generators
- Guarded commands: motivation, semantics, examples
- Industry angle: controlling side effects, avoiding subtle branch bugs

Expressions: What They Are

- An expression is a syntactic form that can be evaluated to produce a value.
- Key properties that differ across languages:
 - ▶ Evaluation order: which subexpression runs first
 - ▶ Side effects: whether evaluation changes program state
 - ▶ Definedness: whether the language specifies behavior for all cases
 - ▶ Type rules: whether expression types are inferred or annotated
- Common sources of confusion:
 - ▶ Operator precedence vs evaluation order are different concepts
 - ▶ A program can be type-correct but still have unspecified behavior in some languages

Operator Precedence vs Evaluation Order

- Operator precedence decides how an expression is grouped (its parse tree).
- Evaluation order decides when the grouped pieces are executed.
- Example grouping:
 - ▶ $1 + 2 * 3$ groups as $1 + (2 * 3)$ due to precedence
- Evaluation order matters when:
 - ▶ function calls have side effects
 - ▶ operands update shared state
 - ▶ short-circuit logic can skip work

Evaluation order: left to right example

TypeScript evaluates function arguments from left to right:

```
1 function log(label: string, value: number): number {  
2     console.log(label, value);  
3     return value;  
4 }  
5 let x = 1;  
6 let y = log("first", x++)  
7         + log("second", x++);  
8 // Output:  
9 // first 1  
10 // second 2  
11 // After expression: x is 3, y is 3
```

Evaluation order: C can be tricky

In C, order of evaluation of subexpressions is often not specified:

```
1  #include <stdio.h>
2  int inc(const char *label, int *p) {
3      printf("%s %d\n", label, *p);
4      return (*p)++;
5  }
6
7  int main(void) {
8      int x = 1;
9      int y = inc("first", &x) + inc("second", &x);
10     printf("x = %d, y = %d\n", x, y);
11 }
```

- The language specification does not guarantee which inc call happens first
- Compilers may choose different orders
- Code that depends on a particular order is fragile and may be incorrect

Side effects

- A side effect is any observable change in program state besides returning a value
 - ▶ Modifying a variable or object field
 - ▶ Writing to a file or network
 - ▶ Printing to the console
- Expressions with side effects can be harder to reason about
 - ▶ Value depends on when and how many times the expression is evaluated
- Many languages allow side effects almost anywhere
 - ▶ Assignment as an expression in C, C++
 - ▶ Function calls with side effects in any language
- Some styles and languages try to limit side effects
 - ▶ Pure functions, functional programming, immutable data

Side effects inside expressions

TypeScript example combining value and side effect:

```
1 let arr = [1, 2];
2
3 function pushAndSum(value: number): number {
4     arr.push(value);           // side effect
5     return arr.reduce((a, b) => a + b, 0);
6 }
7
8 let total = pushAndSum(3) + pushAndSum(4);
9
10 // After expression:
11 // arr is [1, 2, 3, 4]
12 // total is 6 + 10 = 16
```

- The result depends on the order and number of calls
- Avoid hidden side effects inside complex expressions when possible

Short-circuit boolean evaluation

- Boolean operators and and or in many languages are short-circuiting
 - ▶ In a and b, if a is false, b is not evaluated
 - ▶ In a or b, if a is true, b is not evaluated
- This is useful for
 - ▶ Avoiding unnecessary work
 - ▶ Avoiding errors when later expressions would be unsafe
- It also means that boolean expressions can control side effects
 - ▶ Second expression may or may not run

Short-Circuit as a Safety Guard (TypeScript)

```
1 type User = { name: string } | null;
2
3 function greet(u: User) {
4     if (u !== null && u.name.length > 0) {
5         console.log("hello", u.name);
6     } else {
7         console.log("no user");
8     }
9 }
```

- The second condition is only evaluated when `u` is not null
- This is one of the most common and safe uses of short-circuit

Short-Circuit as a Safety Guard (Python)

```
1 def greet(u):  
2     # u is either None or a dict with "name"  
3     if u is not None and len(u["name"]) > 0:  
4         print("hello", u["name"])  
5     else:  
6         print("no user")
```

- The and operator short-circuits like &&
- This avoids exceptions when u is None

Short-Circuit with Side Effects: Pitfall

Short-circuit can hide control flow inside expressions.

```
1 // typescript
2 let connected = false;
3 function connect(): boolean {
4     console.log("connecting...");
5     connected = true;
6     return connected;
7 }
8 // works, but the intent is not obvious
9 connected || connect();
```

Guideline:

- Prefer an explicit if statement when the right side has side effects

Short-Circuit Return Values

In TypeScript / JavaScript:

- `A && B` returns either `A` or `B` (not necessarily a boolean)
- `A || B` returns either `A` or `B`

In Python:

- *and* and *or* also return one of the operands

Consequence:

- This enables idioms like default values
- It can also cause bugs when you expected a boolean

Default Values with Short-Circuit (TypeScript)

```
1 function label(name: string | undefined) {  
2     let shown1 = name || "anonymous";  
3     let shown2 = name ?? "anonymous"; // nullish coalescing  
4  
5     console.log({ shown1, shown2 });  
6 }  
7  
8 label(undefined);  
9 label("");
```

- `||` treats empty string as falsey
- `??` only replaces null or undefined

Default Values with Short-Circuit (Python)

```
1 def label(name):  
2     shown = name or "anonymous"  
3     print(shown)  
4  
5 label(None)  
6 label("")
```

Note:

- Empty string behaves like false in Python
- If empty string is a valid value, use an explicit check instead

Assignment Models

Assignment connects a name to a value (or updates a storage location).

Key questions:

- Is assignment a statement, an expression, or both
- Does assignment copy a value or copy a reference
- What is the type behavior (implicit conversions, coercions)
- Are there compound assignments and destructuring forms

Assignment as an Expression (C, TypeScript)

In C and JavaScript, assignment produces a value.

```
1  #include <stdio.h>
2
3  int main(void) {
4      int x = 0;
5      int y = (x = 5); // assignment yields 5
6      printf("x=%d y=%d\n", x, y);
7      return 0;
8  }
```

Implication:

- compact code is possible
- bugs happen when `=` is confused with `==`

Assignment is a Statement (Python)

In Python, assignment is a statement, not a general expression.

```
1 | x = 5
2 | # y = (x = 5)    # syntax error in Python
3 | y = x
4 | print(x, y)
```

Benefit:

- fewer accidental assignments inside conditions

Related feature:

- Python has a dedicated assignment expression operator `:=` for limited cases

Compound Assignment

Compound assignment updates a location using an operator.

```
1 | let x = 10;  
2 | x += 5;    // x = x + 5  
3 | x *= 2;    // x = x * 2  
4 | console.log(x);
```

Notes:

- C and TypeScript support many compound operators
- Python also supports them (`+=`, `*=`, etc.)
- With objects, `+=` may mutate or allocate depending on language and type

Chained Assignment: Readability vs Risk

```
1 // typescript
2 let a: number, b: number, c: number;
3 a = b = c = 0;
4 console.log(a, b, c);
```

```
1 # python
2 a = b = c = 0
3 print(a, b, c)
```

- Works well for immutable values (numbers, strings)
- Can be surprising for mutable objects if all names reference the same object

Chained Assignment with Mutability (Python)

```
1 | a = b = []  
2 | a.append(1)  
3 | print(a)  # [1]  
4 | print(b)  # [1]
```

- a and b refer to the same list object
- mutation through one name is observed through the other

Safer pattern:

- a = [] ; b = []

Destructuring and Multiple Assignment

TypeScript destructuring:

```
1 let pair: [number, string] = [7, "seven"];
2 let [n, s] = pair;
3
4 let obj = { x: 1, y: 2 };
5 let { x, y } = obj;
6
7 console.log(n, s, x, y);
```

Python tuple unpacking:

```
1 n, s = (7, "seven")
2 x, y = [1, 2]
3 print(n, s, x, y)
```

Assignment in conditions: what happens?

C code:

```
1  int x = 5;
2  if (x = 0) {
3      printf("x is zero\n");
4  } else {
5      printf("x is nonzero\n");
6  }
```

Questions to discuss:

- What does $x = 0$ do to the value stored in x ?
- What is the value of the expression $x = 0$?
- Which branch runs, and what is printed?
- How would you write the intended version in C?

Compare with Python:

```
1  x = 5
2
3  # if x = 0:           # syntax error in Python
4  #     print("x is zero")
```

Sequences and Blocks

- A sequence is a way to express multiple computations in order.
- A block groups statements into a single syntactic unit:
 - ▶ often introduces a scope
 - ▶ often controls lifetime of locals
 - ▶ often used after if/while/for
- Different languages emphasize different mechanisms:
 - ▶ C, TypeScript: braces define blocks; semicolons separate statements
 - ▶ Python: indentation defines blocks; newline separates statements

Blocks and scope

```
1 //typescript
2 let x = 1;
3 {
4     let x = 2;           // inner x shadows outer x
5     console.log(x);      // prints 2
6 }
7 console.log(x);          // prints 1
```

```
1 # python
2 x = 1
3 if True:
4     x = 2                # modifies same x
5 print(x)                 # prints 2
```

- In TypeScript, block scope with `let` and `const`
- In Python, `if` and `while` blocks do not introduce a new local scope

Selection Constructs

Selection chooses a control-flow path based on conditions or patterns.

Common forms:

- if / else if / else
- switch (C, TypeScript) with cases
- match (Rust), match/case (Python) with pattern matching

Design choices:

- fall-through vs no fall-through
- exhaustiveness checking (compile-time guarantees)
- pattern matching richness (values, types, structure)

Selection: if statements

- if statements choose between blocks based on a boolean condition
- General forms
 - ▶ Single branch if (condition) block
 - ▶ Two-way if (condition) block else block
 - ▶ Multi-way if, else if, else chains
- Conditions are often expressions that evaluate to boolean
 - ▶ Some languages allow non-boolean values used as truthy or falsy
- Nesting ifs leads to many possible paths, which can be a source of bugs

If / Else: Basic Pattern (TypeScript)

```
1 function category(score: number): string {  
2     if (score >= 90) {  
3         return "A";  
4     } else if (score >= 80) {  
5         return "B";  
6     } else if (score >= 70) {  
7         return "C";  
8     } else {  
9         return "F";  
10    }  
11 }
```

- ordered conditions implement a priority
- readability improves when conditions are simple and side-effect free

If / Else: Common Bug Pattern

Confusing assignment with equality is a classic error in languages where assignment is an expression.

```
1 // C
2 int ok = 0;
3 /* bug: assigns 1 to ok, condition becomes true */
4 if (ok = 1) {
5     /* ... */
6 }
```

Safer patterns:

- enable compiler warnings
- prefer explicit comparisons: `ok == 1`
- keep assignments out of conditions

Multi-way selection: switch and match

- When there are many discrete cases, if chains become verbose
- switch in C, C++, Java, TypeScript
 - ▶ Compares a value against several constant cases
 - ▶ Typically allows fallthrough unless break is used
- match expressions in some languages, for example Rust
 - ▶ Use pattern matching
 - ▶ Usually are expressions that produce a value
 - ▶ Often must be exhaustive: every case handled

Switch in C

```
1  int day = 3;
2  switch (day) {
3      case 1:
4          printf("Monday\n");
5          break;
6      case 2:
7          printf("Tuesday\n");
8          break;
9      case 3:
10     case 4:
11         printf("Midweek\n");
12         break;
13     default:
14         printf("Other day\n");
15         break;
16 }
```

- Without break, execution would fall through to the next case
- fallthrough can be useful but is also a common source of bugs

Switch in TypeScript: Similar Shape, Different Use

```
1 function httpMessage(code: number): string {  
2     switch (code) {  
3         case 200: return "OK";  
4         case 404: return "Not Found";  
5         case 500: return "Server Error";  
6         default: return "Unknown";  
7     }  
8 }
```

Notes:

- returning directly avoids forgetting break
- for complex logic, match-like patterns can be clearer

Pattern Matching: Motivation

Pattern matching lets you select branches by structure, not only by boolean conditions.

Benefits:

- often more readable than chained if/else
- encourages explicit handling of all cases
- can destructure data while branching

Languages:

- Rust: match is central and exhaustiveness-checked
- Python: match/case supports structural patterns (3.10+)
- TypeScript: common alternative is discriminated unions plus switch

Rust match as an expression

```
1 fn classify_score(score: i32) -> &'static str {  
2     match score {  
3         0..=59 => "fail",  
4         60..=79 => "pass",  
5         80..=89 => "good",  
6         90..=100 => "excellent",  
7         _ => "invalid",  
8     }  
9 }
```

- match must handle all possible patterns, or use a catch all pattern
- match returns a value, so it is an expression, not just a statement
- Range patterns and other pattern forms make multi-way selection concise

Python match/case: Structural Patterns

```
1 def handle(m):  
2     match m:  
3         case ("move", x, y):  
4             return x + y  
5         case ("write", s):  
6             return len(s)  
7         case ("quit",):  
8             return 0  
9         case _:  
10            return -1
```

Notes:

- patterns can match shapes such as tuples and literals
- the wildcard case `_` acts like default

Iteration Constructs

Iteration repeats a computation.

Common forms:

- while: repeat while condition holds
- for: counting loop or general loop with explicit control
- foreach: iterate over elements of a collection
- iterators: abstract the notion of producing next elements

Typical design choices:

- where the loop variable is scoped
- whether the language exposes iterators directly
- how termination is represented (condition, StopIteration, option type)

while Loop: Basic Form (TypeScript)

```
1 let n = 5;  
2 let acc = 1;  
3  
4 while (n > 1) {  
5     acc *= n;  
6     n--;  
7 }  
8  
9 console.log(acc);
```

- ensure the loop makes progress toward termination
- keep mutation localized and easy to see

For loops with counters

C style:

```
1 | int sum = 0;
2 |
3 | for (int i = 1; i <= 10; i++) {
4 |     sum += i;
5 | }
```

TypeScript style:

```
1 | let sum = 0;
2 |
3 | for (let i = 1; i <= 10; i++) {
4 |     sum += i;
5 | }
```

- Initialization, condition, and update are bundled together
- Helps avoid forgetting to update the loop variable

foreach: Iterating Collections (Python)

```
1 nums = [1, 2, 3, 4]
2 total = 0
3
4 for x in nums:
5     total += x
6
7 print(total)
```

Notes:

- for in Python is a foreach over an iterator
- loop variable remains in scope after the loop

foreach: Iterating Collections (TypeScript)

```
1 let nums = [1, 2, 3, 4];  
2 let total = 0;  
3  
4 for (const x of nums) {  
5     total += x;  
6 }  
7  
8 console.log(total);
```

Notes:

- for..of iterates values from an iterable
- for..in iterates property names (different meaning)

Iterators conceptually

- An iterator is an abstraction that produces a sequence of values
 - ▶ Has an internal state
 - ▶ Each call to next gives the next value or signals the end
- Many languages define iterator protocols
 - ▶ for loops can work with any object that implements the protocol
- Iterators separate how to produce values from how to consume them
 - ▶ Makes code more modular and testable

Iterator Protocol in TypeScript

JavaScript defines an iterator protocol using *Symbol.iterator*.

```
1 function range(n: number) {  
2     return {  
3         [Symbol.iterator]() {  
4             let i = 0;  
5             return {  
6                 next() {  
7                     if (i < n) return { value: i++, done: false };  
8                     return { value: undefined, done: true };  
9                 }  
10            };  
11        }  
12    };  
13 }  
14 for (const x of range(4)) {  
15     console.log(x);  
16 }
```

- iterators separate producing values from consuming them. *done* signals termination

Generators in TypeScript

Generators provide a compact way to write iterators.

```
1 function* range(n: number) {  
2     for (let i = 0; i < n; i++) {  
3         yield i;  
4     }  
5 }  
6  
7 for (const x of range(4)) {  
8     console.log(x);  
9 }
```

Notes:

- yield produces values one at a time
- control resumes after yield on the next iteration step

Iterators in Python

Python uses `iter()` and `next()` under the hood for `for`-loops.

```
1 | it = iter([10, 20, 30])  
2 | print(next(it))  
3 | print(next(it))  
4 | print(next(it))
```

Notes:

- when exhausted, `next` raises `StopIteration`
- `for` handles `StopIteration` automatically

Guarded Commands (Dijkstra): Motivation

Guarded commands are a control model designed to emphasize correctness.
Core idea:

- replace single-condition branching with a set of guarded alternatives
- if multiple guards are true, any may be chosen
- if no guard is true, the command is blocked (or fails)

Why it is interesting in a programming languages course:

- it separates the notion of enabling conditions from chosen actions
- it connects well to reasoning about programs (preconditions and invariants)

Guarded Commands: Syntax (Pseudo-Code)

Selection (if-like):

- if
 - ▶ $G1 \rightarrow S1$
 $G2 \rightarrow S2$
...
 $Gn \rightarrow Sn$
- fi

Iteration (while-like):

- do
 - ▶ $G1 \rightarrow S1$
 $G2 \rightarrow S2$
...
 $Gn \rightarrow Sn$
- od

Meaning:

- G_i is a boolean guard
- S_i is a statement sequence

Guarded Selection: Semantics

For guarded if:

- Evaluate which guards are true
- If at least one guard is true, choose one true guard and execute its statement
- If no guards are true, the command has no enabled action

Why this matters:

- it avoids committing to an arbitrary priority order
- it forces you to consider completeness: do you cover all cases

How languages simulate this:

- explicit if/else chains with carefully designed conditions
- match with guards (Rust) or case guards (Python) in structured forms

Guarded commands example: Absolute Value

Pseudo code :

```
1  if  $x < 0$   ->  $x := -x$ 
2  []  $x == 0$  -> skip
3  []  $x > 0$   ->  $x := x$ 
4  fi
```

- exactly one guard is true for any integer x
- completeness is easy to argue

Guarded Iteration: Semantics

For guarded do:

- While at least one guard is true, choose one true guard and execute its statement
- Stop when all guards are false

Reasoning style:

- pick an invariant that remains true after any enabled action
- prove termination by showing progress toward a state where all guards are false

This viewpoint connects to concurrency:

- multiple guards can represent different enabled actions in a system

Guarded commands vs mainstream control flow

- Guarded commands emphasize correctness reasoning
 - ▶ Loop invariants and termination arguments
- Mainstream languages use deterministic if, switch, while, for
- Conceptually, guarded commands can be compiled into standard control flow
 - ▶ Choose one true guard in some fixed order
 - ▶ Use usual if and while constructs
- Studying guarded commands helps understand the structure of control flow

Industry angle: controlling side effects

- Side effects in expressions can make code hard to understand
 - ▶ Hidden mutations, unexpected I/O, global state changes
- Common practices
 - ▶ Keep expressions mostly pure, side effects in clearly marked statements
 - ▶ Avoid assignment inside conditions when possible
 - ▶ Limit the scope of variables and mutable state
- Functional style in TypeScript, Python, and other languages
 - ▶ Use map, filter, reduce instead of manual loops when appropriate
- Tests and code review often focus on surprising side effects

Avoiding subtle branch bugs

Problematic C code:

```
1 | if (x = 0) {  
2 |     printf("zero\n");  
3 | }
```

Intended:

```
1 | if (x == 0) {  
2 |     printf("zero\n");  
3 | }
```

- In C, $x = 0$ is an expression that evaluates to 0
- The condition is always false, and x is modified

Safer styles:

- Use comparisons in conditions, not assignments
- In languages where assignment is only a statement, this bug cannot occur
- Use linters and compiler warnings to catch suspicious constructs

Summary

- Evaluation order and side effects strongly influence program behavior
- Short-circuit boolean operators control both work and safety
- Assignments, sequences, and blocks are the foundation of imperative code
- Selection and iteration constructs shape control flow
- Iterators provide a higher level view of looping
- Guarded commands offer an abstract model to reason about programs
- In practice, careful handling of side effects and branches reduces bugs